



Tecnológico de Monterrey

INSTITUTO TECNOLÓGICO Y DE ESTUDIOS
SUPERIORES DE MONTERREY

CAPSTONE PROJECT (TEAM #49)

Executive Summary

Author Julia Gabriela Pinedo A01795315

Professor Dr. Grettel Barceló Alonso

Project Advisor Dr. Guillermo Mota Medina

November 9, 2025

Contents

1	Problem Overview	3
1.1	Background	3
1.2	Problem Statement	3
1.3	Justification	4
1.4	Objectives	5
1.5	Data Description	5
2	Key Findings	7
2.1	Data Integrity	7
2.2	Numerical and Categorical Variables	8
2.3	Feature Distributions	8
2.4	Correlations	9
2.5	Bivariate Relationships	10
2.6	Class Balance	11
2.7	Baseline Metrics	11
3	Model	13
3.1	Candidate Models	13
3.2	Final Model and Justification	14
4	Recommendations	16
5	Cost-Benefit Assessment	17
5.1	Incurred Costs	17
5.2	Expected Costs	17
5.3	Potential Benefits	18
5.4	Potential Risks	19
6	References	20

1 Problem Overview

1.1 Background

The **box-pushing problem** has become a crucial benchmark in cooperative robotics, as it models scenarios where one or more robots must push a heavy box toward a goal region while avoiding obstacles and walls [1]. This task demands superior coordination between robots, which has led to its adoption in **Multi-Agent Reinforcement Learning (MARL)**. MARL is the paradigm for fully cooperative tasks in which agents must share intent, avoid interference, and learn complementary roles [2][3].

Recent studies indicate that Multi-Agent Reinforcement Learning research often uses the box-pushing problem, underscoring its value for evaluating coordination and robustness of models [2][3]. However, some authors note that canonical MARL-based approaches may underperform in comparison to **single-agent solutions**, due to coordination failures and credit assignment challenges. This observation motivates the exploration of methods that enhance cooperative learning without relying on excessively complex environments or computationally expensive strategies.

Based on this premise, there is a significant need to develop a project that not only establishes a **Single-Agent Reinforcement Learning** baseline, but also guides the development of a multi-agent solution capable of addressing these limitations. Although the current exploration remains within an academic context, the problem remains relevant because it arises from real-world needs and contributes to the *long-term development* of more resilient, efficient, and autonomous robotic solutions capable of operating in complex scenarios or environments [4].

1.2 Problem Statement

Multi-robot formation control has been extensively studied because of its importance in scenarios where multiple robots must collaborate to move a payload that is too heavy or bulky for one robot. In such *self-organizing systems* (**Figure 1.1**), the agents need to operate in coordination to avoid conflicts such as inefficient paths, counterproductive forces that result in little or no movement, or impeded learning progress. Since this problem involves one or more robots working together, it is connected to the **multi-robot box pushing problem**, a classic cooperative multi-agent task in robotics, where one or more robots must push a box toward a goal region while avoiding obstacles [5][6].

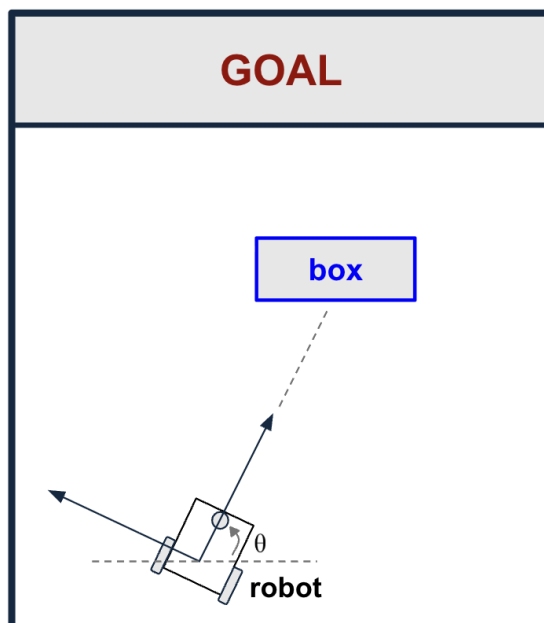


Figure 1.1 Diagram of the classic paradigm of the box-pushing problem.

The key challenges of this problem include:

- Enabling robots to find efficient paths and coordinate their pushes.
- Planning and executing rotational maneuvers to avoid obstacles in the environment.
- Supporting adaptive decision-making as the “*state*” (positions of robots and box) continuously changes.

While multi-agent solutions are conceptually more scalable and robust, in practice, **single-agent reinforcement learning** approaches often demonstrate a superior performance in terms of efficiency and reliability, whereas simple MARL implementations generally exhibit limitations [2].

This gap underscores the need to **improve collaborative learning in multi-agent systems** applied to box-pushing scenarios. Establishing a single-agent reinforcement learning **baseline** is essential to highlight the limitations of current MARL methods and provide a clearer benchmark for developing more robust multi-agent solutions.

1.3 Justification

Addressing this problem through a Multi-Agent Reinforcement Learning approach is essential, as it represents a **fundamental challenge in collaborative robotics** and provides an experimental framework for analyzing the effectiveness of multi-robot collaboration. However, before moving toward a multi-agent environment, it is necessary to **establish a**

solid baseline with a single agent. This is to understand our current limitations and lay the groundwork for designing more robust cooperative strategies.

From an academic perspective, this project provides a way to evaluate coordination strategies within Reinforcement Learning. Demonstrating a successful performance in the **Single-Agent Reinforcement Learning (SARL)** domain will generate a reliable reference point, facilitating its subsequent expansion to multi-agent environments and contributing to the development of scalable learning architectures capable of handling coordination and communication in complex robotic systems.

1.4 Objectives

- Establish a Single-Agent Reinforcement Learning baseline.
- Extend the same SARL baseline algorithm to a multi-agent setting.
- Design and implement a robust Multi-Agent Reinforcement Learning framework that enables effective cooperation among multiple differential-drive robots in simulated environments.
- Integrate a custom differential-drive robot model equipped with odometry, IMU (*Inertial Measurement Unit*), and contact detection sensors to support a realistic RL environment in Gazebo + ROS 2.
- Evaluate performance across key metrics such as: success rate, efficiency, robustness under agent failures, and adaptability to obstacles.

1.5 Data Description

Since this project is based on Reinforcement Learning (RL) techniques, we do not rely on a traditional static dataset. Instead, the data is generated through interaction with the simulation in Gazebo + ROS 2. The environment consists of:

- A **differential-drive robot model** with defined physical dimensions, sensors (odometry, IMU, contact detection), and control inputs (**Table 1.1**).

Table 1.1 Robot dimensions.

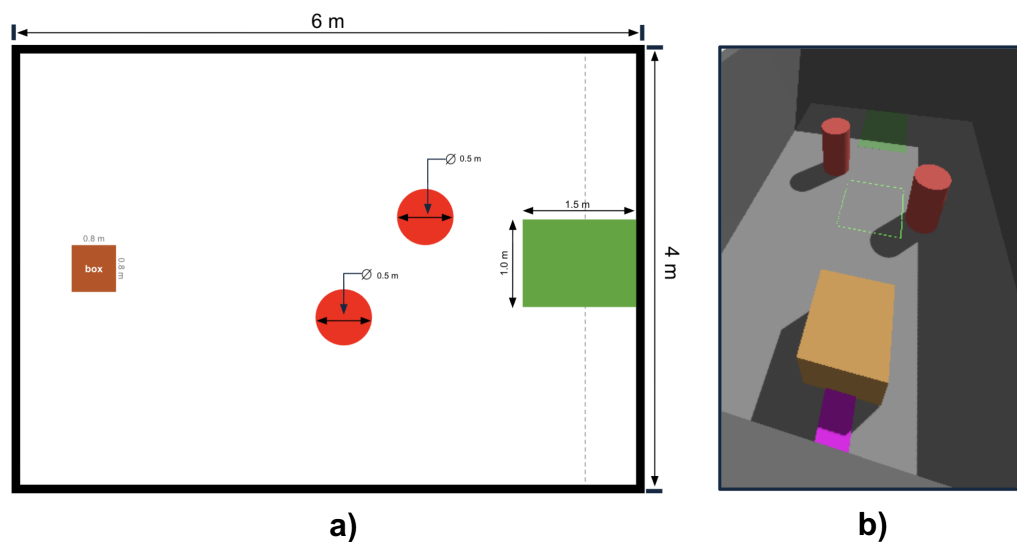
Part	Dimensions
Base (main body)	Length: 0.55 m Width: 0.30 m Height: 0.12 m Mass: ~ 7.5 kg
Wheels (x2)	Radius: 0.08 m Width: 0.03 m Mass: 0.3 kg
Caster	Radius: 0.08 m Mass: 0.2 kg

- A **box** with fixed dimensions and mass, serving as the shared object to be pushed (Table 1.2).

Table 1.2 Box dimensions.

Part	Dimensions
Box (object)	$0.80\text{ m} \times 0.80\text{ m} \times 0.80\text{ m}$ Mass: ~ 7.5 kg

- A **simulation arena**, including floor surface properties (friction coefficient) and placement of static obstacles.

**Figure 1.2 Simulation environment. a) Diagram of the arena, b) Arena modeled and simulated in Gazebo.**

2 Key Findings

To conduct the **Exploratory Data Analysis (EDA)**, a manual simulation of the environment was run. Each episode corresponds to a simulated full run of the diff-drive robot inside the Gazebo arena, capturing sensor readings, distances to obstacles and goal regions, and discrete event flags [7].

2.1 Data Integrity

For the first run, a null-value analysis was performed to assess the data completeness and integrity (**Figure 2.1**).

```

RangeIndex: 500 entries, 0 to 499
Data columns (total 33 columns):
#   Column                Non-Null Count  Dtype
---  -
0   episode                500 non-null    int64
1   step                  500 non-null    int64
2   done                  500 non-null    bool
3   done_reason           500 non-null    object
4   robot_x                500 non-null    float64
5   robot_y                500 non-null    float64
6   robot_yaw              500 non-null    float64
7   robot_vx               500 non-null    float64
8   robot_vy               500 non-null    float64
9   robot_wz               500 non-null    float64
10  imu_wx                 500 non-null    float64
11  imu_wy                 500 non-null    float64
12  imu_wz                 500 non-null    float64
13  imu_ax                 500 non-null    float64
14  imu_ay                 500 non-null    float64
15  imu_az                 500 non-null    float64
16  box_x                  500 non-null    float64
17  box_y                  500 non-null    float64
18  box_yaw                500 non-null    float64
19  goal_x                 500 non-null    float64
20  goal_y                 500 non-null    float64
21  contact_flag           500 non-null    bool
22  cmd_vx                 500 non-null    float64
23  cmd_wz                 500 non-null    float64
24  wheel_l_rad_s          500 non-null    float64
25  wheel_r_rad_s          500 non-null    float64
26  r_progress              500 non-null    float64
27  r_collision             500 non-null    float64
28  r_efficiency            500 non-null    float64
29  r_rot_pen              500 non-null    float64
30  r_goal                 500 non-null    float64
31  r_total                500 non-null    float64
32  in_goal                500 non-null    bool
dtypes: bool(3), float64(27), int64(2), object(1)

```

Figure 2.1 Table with all the entries and non-null counts.

The results showed **zero missing entries** across all variables, demonstrating a complete and consistent data capture and exhibiting high structural integrity and reproducibility. This improvement was achieved after refining the ROS 2 + Gazebo bridge configuration, which synchronized the publishing rates and message buffering between the sensors and the robot's control node. The enhanced communication pipeline eliminated message dropouts

and ensured full temporal alignment of sensor readings. The data provides a realistic yet deterministic basis for defining the state representation.

2.2 Numerical and Categorical Variables

The dataset contains **31 columns and 500 entries**, all fully populated. Variables will later be distributed into three main categories according to their role in the reinforcement learning process: state observation, action command, and reward feedback.

Numerical Variables (27 features). These variables describe continuous magnitudes related to the robot’s kinematics, inertial sensor, spatial configuration of the environment, and reward components (**Table 2.1**).

Table 2.1 Description of the numerical variables.

Type	Variables
Spatial and Orientation	robot_x, robot_y, robot_yaw, box_x, box_y, box_yaw, goal_x, goal_y
Kinematics	robot_vx, robot_vy, robot_wz, wheel_l_rad_s, wheel_r_rad_s
Inertial Features (IMU)	imu_wx, imu_wy, imu_wz, imu_ax, imu_ay, imu_az
Commands	cmd_vx, cmd_wz
Reward Components	r_progress, r_collision, r_efficiency, r_rot_pen, r_goal, r_total

Categorical and Boolean Variables (4 features). This is a smaller subset of variables encoding discrete events and logical states (**Table 2.2**).

Table 2.2 Description of the categorical variables.

Type	Variables
Episode Tracking	episode, step (integer)
Event Flags	contact_flag, in_goal, done (boolean)
Outcome Label	imu_ax, imu_ay, imu_az
Commands	done_reason (object/string)

2.3 Feature Distributions

To characterize the variability of the simulated environment, the main continuous variables were visualized as histograms of spatial, kinematic, and inertial features for both the robot and the pushable box. Each feature distribution describes how often specific values

occur throughout all episodes, revealing the range of motion, sensor behavior, and physical consistency of the simulation (**Figure 2.2**).

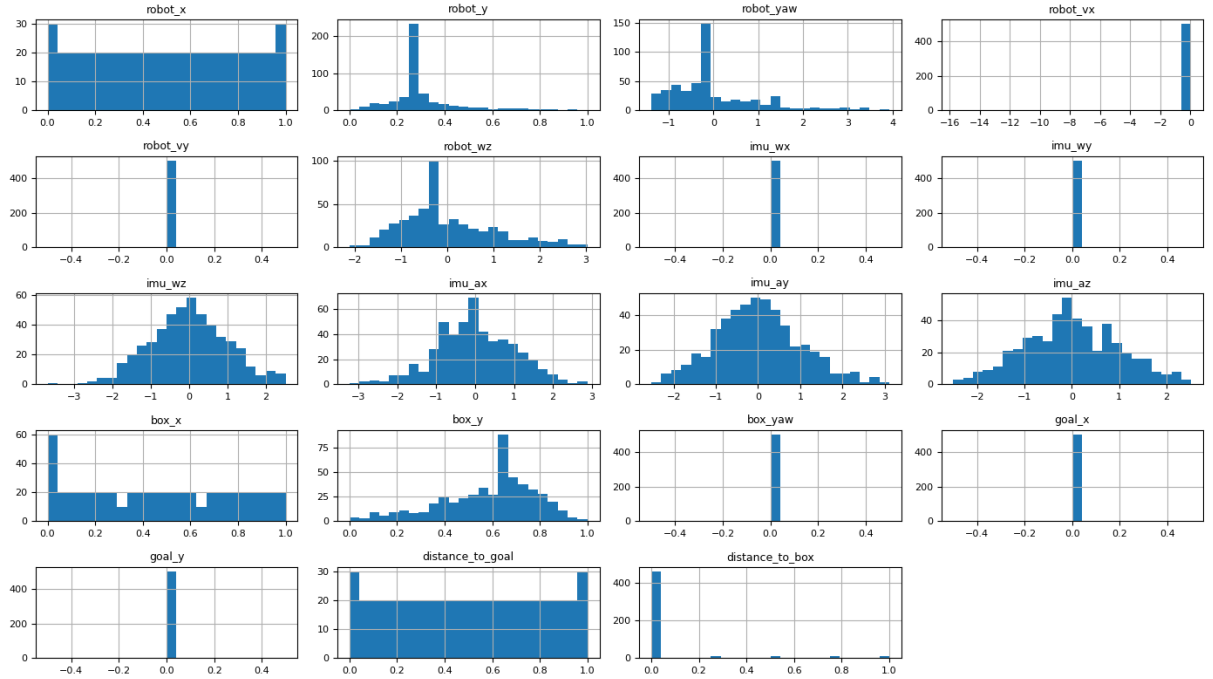


Figure 2.2 Histograms generated with the main continuous variables.

These **continuous features** show a *stable and consistent* distribution, aligned with the simulation’s expected behavior. Spatial and orientation variables confirm a consistent navigation within the simulated environment, while motion-related features reflect smooth trajectories and controlled maneuvering. On the other hand, inertial readings exhibit symmetric distributions, indicating a realistic sensor behavior. Overall, the distributions verify that the setup produces accurate, synchronized, and noise-consistent sensor data.

2.4 Correlations

The **correlation matrix** shows a clear dependency between the robot’s positional, kinematic, and sensory variables, validating the physical coherence of the simulated environment (**Figure 2.3**). Strong positive correlations indicate that the lateral position and rotation are intrinsically linked to turning maneuvers. On the other hand, negative correlations confirm that moving forward along the x-axis effectively reduces the goal distance. In general, these relationships confirm that the simulation ensures motion and sensor data evolve consistently with the robot’s physical interactions.

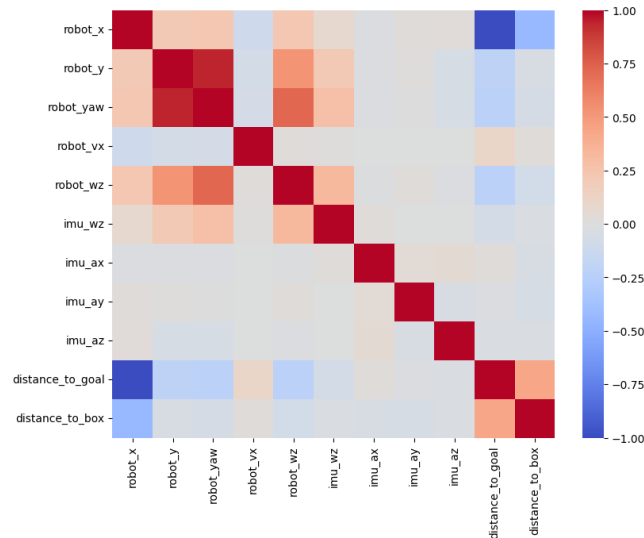


Figure 2.3 Correlation matrix.

2.5 Bivariate Relationships

The **bivariate relationships** further illustrate how motion dynamics influence the task outcomes (Figure 2.4).

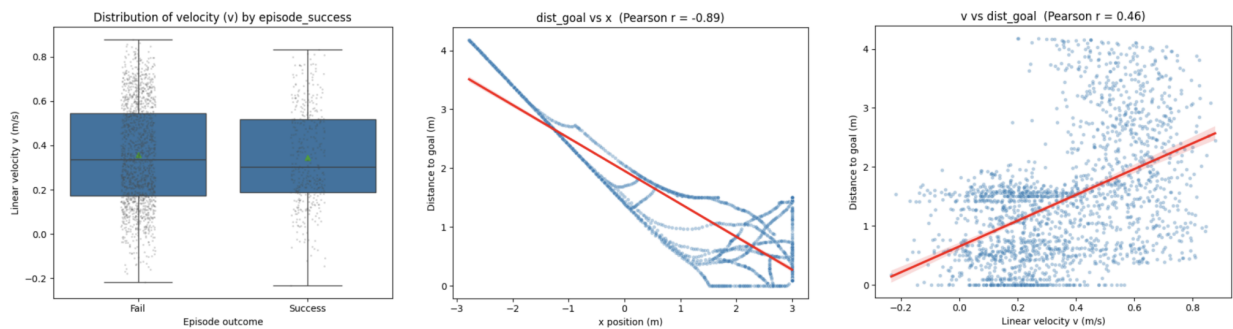


Figure 2.4 Bivariate relationships between key motion and task variables.

- The comparison of linear velocity across successful and failed episodes suggests that the success depends more on forward movement consistency than on higher speed.
- The scatter plots reveal that as the robot's x-position increases, the distance to the goal decreases nearly linearly, confirming that forward motion along this axis is the primary dimension of task progress.
- A moderate positive correlation between velocity and distance to the goal indicates that higher speeds occur when the robot is far from the target.

These findings emphasize that effective navigation arises from stable and progressive motion rather than abrupt acceleration.

2.6 Class Balance

The categorical variables analyzed correspond to the event flags *contact_flag* and *in_goal*, which represent key interactions during each episode. The results show that the robot made contact with the box in most steps, but successfully reached the goal region in only **4 out of 500 observations** (Figure 2.5).

contact_flag	Counts:
True	448
False	52

in_goal	Counts:
False	496
True	4

Figure 2.5 Table with the count of the *contact_flag* and *in_goal* variables.

This strong imbalance was expected in the first simulation without a trained policy, as the agent's movements were governed by random and manual commands rather than learned behavior. The predominance of unsuccessful outcomes accurately reflects the task's inherent difficulty in the environment.

2.7 Baseline Metrics

To evaluate the performance of the model, the following core evaluation metrics were defined:

- **Episode return over time (learning curve):**

This measures whether the policy improves its ability to maximize cumulative rewards, reflecting how effectively the robot learns to push the box toward the goal as training progresses.

- **Success rate (goal completion):**

Captures the fraction of episodes in which the box reaches the goal region, serving as a direct and interpretable indicator of task achievement independent of reward shaping.

- **Episode length/steps to success (efficiency):**

Evaluates how quickly the robot completes successful episodes, with shorter trajectories indicating higher operational efficiency.

The selected **floor criteria** adopt the evaluation success rate (*eval_success_rate*) as the principal metric, since it measures task completion under evaluation conditions and remains robust to variations in reward shaping (**Table 2.3**).

Table 2.3 Floor criteria.

Metric
Eval success rate ≥ 0.20
Median steps-to-success ≤ 80
Positive learning trend on eval (rolling/local slope > 0)
No overfitting signal (<i>eval_return</i> – <i>train_return</i> at eval points) ≥ -1.0

3 Model

3.1 Candidate Models

Algorithm	Policy regime	Typical action space	Key strengths	Key weaknesses
Monte Carlo Policy Gradient (REINFORCE)	On-policy	Discrete/continuous	Simple, direct gradient on returns, easy to prototype.	Very high variance, slow convergence, sensitive to reward shaping.
Deep Q-Network (DQN)	Off-policy	Discrete	Sample-efficient via replay, stable with target network, mature tooling.	Can struggle with non-stationarity, overestimation, needs careful target tuning.
Distributional DQN (C51)	Off-policy	Discrete	Rich signal, often faster, stable learning.	More hyperparams, implementation complexity, gains can be environment dependent.
Soft Actor-Critic (SAC)	Off-policy	Continuous (discrete variants exist)	Excellent exploration via entropy, very stable in continuous control, reuses data efficiently.	Discrete adaptations add complexity, extra critics/temperature to tune.
Advantage Actor-Critic (A2C)	On-policy	Discrete/continuous	Low-variance updates via advantage, simple, stable, works well with eval checkpoints.	Less sample reuse than off-policy, could need fine LR/GAE tuning.
Proximal Policy Optimization (PPO)	On-policy	Discrete/continuous	Robust via clipped updates, strong general performance, forgiving hyperparams.	More compute per update, still-on policy (less sample reuse).

Figure 3.1 Table with the comparison of the six models.

To conduct the **Exploratory Data Analysis (EDA)**, a manual simulation of the environment was run. Each episode corresponds to a simulated full run of the diff-drive robot inside the Gazebo arena, capturing sensor readings, distances to obstacles and goal regions, and discrete event flags [7][8][9][10].

Algorithm	Eval success rate ≥ 0.20	Median steps-to-success ≤ 80	Positive learning trend on eval (rolling/local slope > 0)	No overfitting signal (eval_return - train_return at eval points) ≥ -1.0	Execution time (mins)
Monte Carlo Policy Gradient (REINFORCE)	0.35	62.5 steps	0.176	0.382	6.42
Deep Q-Network (DQN)	0.35	60 steps	0.083	1.335	6.37
Distributional DQN (C51)	0.35	73 steps	-0.002	-0.726	6.32
Soft Actor-Critic (SAC)	0.35	58 steps	-0.124	0.921	6.38
Advantage Actor-Critic (A2C)	0.76	57 steps	2.614	0.9502	6.11
Proximal Policy Optimization (PPO)	0.68	58 steps	2.444	0.9504	6.22

Figure 3.2 Table with the comparison of the six models.

The results indicate that **A2C** achieved the most consistent performance, combining a high evaluation success rate and efficient trajectories, while maintaining stability between training and evaluation returns. **PPO** achieved comparable results, but required longer training and higher computational costs. In contrast, value-based methods such as DQN and C51 plateaued at lower success rates and displayed minimal or negative learning slopes, indicating limited improvement under the same simulation budget.

3.2 Final Model and Justification

Based on the empirical results of the six candidate models, **Advantage Actor-Critic (A2C)** was selected as the final model due to its superior performance. To further enhance

model stability, a bagging technique was incorporated into the implementation. The following results (**Table 3.1**) summarize the model’s performance after applying A2C + Bagging:

Table 3.1 Performance of final model: Advantage Actor-Critic + Bagging.

Metric	Result
Eval success rate ≥ 0.20	0.79
Median steps-to-success ≤ 80	56 steps
Positive learning trend on eval (rolling/local slope > 0)	0.067
No overfitting signal ($eval_return - train_return$ at eval points) ≥ -1.0	0.127

To visualize the performance of the model on the selected metrics and summarize the learning dynamics of the final model, the following graphs were generated (**Figure 3.3**):

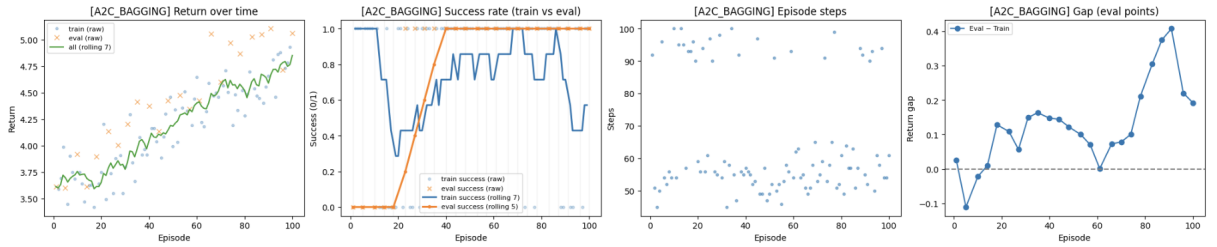


Figure 3.3 Graphs of the A2C Bagging model.

We observe that the performance of the model **meets the floor criteria**, making it strong enough to serve as a reference policy. These results satisfy objectives 1, 4, and 5 outlined in **Section 1** and set the reference standard for objectives 2 and 3, which expect the expansion into a multi-agent setting.

Since the model is stable, reproducible, and reliable, it is acceptable to be **production-ready**, where it can serve as the reference model for the next phase and be reused as a foundation to train, compare, or extend into the multi-agent framework.

4 Recommendations

Regarding the potential for improvement in the model's performance, the following refinements are proposed to strengthen the single-agent RL foundation without modifying the simulation environment:

- Include stronger penalties where errors are most costly to encourage a smoother and more purposeful behavior.
- Implement a tighter *exploration-to-exploitation* schedule, for example, by gradually reducing the entropy bonus to allow the agent to shift from broad exploration toward more consistent, goal-directed actions as training progresses.

Beyond these adjustments, the following recommendations have been established to implement the solution:

1. Standardization of the computational environment. Although the use of Windows Subsystem for Linux has supported initial experiments, migrating to a system with Ubuntu Linux 22.04 for the simulation is recommended for best compatibility. In addition, shipping a single Dockerfile (*CPU + optional CUDA tag*) will guarantee that the runs are reproducible on any machine.

2. Automate and share the analysis pipeline. To make the result visualization transparent and reusable, a Jupyter Notebook should be created to import metrics, generate the plots automatically, the summary table, and save the output figures to ingest them into reports or LaTeX documents for easy tracking.

3. Tighten and stabilize the single-agent baseline. As discussed previously, applying stronger penalties for inefficient or unstable actions and adopting a tighter exploration-to-exploitation schedule will improve training stability. These refinements would also provide a cleaner and more deterministic reference for the future multi-agent cooperation experiments.

5 Cost-Benefit Assessment

5.1 Incurred Costs

The project operates entirely in a research and simulation environment, with all the development performed on a local workstation. Therefore, **no direct financial costs have been incurred** beyond existing personal computational resources. The activities performed across each phase of the CRISP-ML methodology are linked to the following costs (**Figure 5.1**):

CRISP-ML Phase	Description	Incurred/ Estimated Cost (USD)	Notes
Business & Data Understanding	Literature review, simulator design, and environment setup	0	Conducted locally using open-source tools (Gazebo, ROS 2, Python)
Data Preparation	Synthetic data generation through local simulation	0	Data produced via Gazebo without licensing costs
Modeling	Implementation and tuning of the candidate models	0	Executed locally using Python and PyTorch
Evaluation	Model comparison and selection	0	Performed on a local machine
Deployment (preliminary)	Evaluation harness	0	Represents minimal compute energy usage for local runs

Figure 5.1 Detailed costs per CRISP-ML phase.

5.2 Expected Costs

As mentioned in **Section 5.1**, all training and evaluation processes are performed locally. The project does not operate on a continuous 24/7 and the simulations are launched on demand for refinement or reporting. However, **Google Cloud (Vertex AI)** [11] was selected as the target environment for reproducibility and scalability testing. This ensures that when required, the same training and evaluation harness can be executed on the cloud for validation or parallel experimentation (**Figure 5.2**).

Category	Description	Estimated Cost (USD)
Compute (Local workstation)	CPU/GPU usage during training and evaluation	0
Storage	Model checkpoints, logs, Docker images	0
Software	Open-source stack	0
Cloud (Vertex AI)	Optional containerized training/evaluation for reproducibility	20-30 dollars/use (short validation sessions only)

Figure 5.2 Associated costs for the project.

In summary, the project is not an operational deployment but a **research-stage framework** executed only on demand for experimentation, refinement, or data collection. Consequently, the system does *not* require continuous maintenance or monitoring. Once the training run completes and results are logged, the environment can be closed and relaunched whenever new tests are needed.

5.3 Potential Benefits

The primary benefit of the project lies in **establishing a reproducible reinforcement learning benchmark** that can later be replicated in real robotic systems with a MARL setup.

Quantifiable Benefits:

- Faster adaptation from single-agent to multi-agent settings.
- Lower data acquisition costs, as all data is simulated.
- Full compatibility between local environments for reproducible results.

Qualitative Benefits:

- Improved research reproducibility and model interpretability through standardized pipelines.
- Contribution to academic and industrial innovation in intelligent robotic control and multi-agent coordination.
- Establishes a baseline reference policy for comparison in future reinforcement learning research.

5.4 Potential Risks

Although the current project remains in a controlled research environment, several technical and methodological risks were identified to ensure a reliable and reproducible outcome. These risks are characteristics of simulation-based RL rather than a production system (Figure 5.3):

Category	Risk	Description/Potential Impact
Data Risks	Overfitting to the simulated environment	The learned policy may become too specific to one configuration of the arena or reward structure, limiting its adaptability to new layouts or dynamic conditions
Testing & Trust Risks	Non-deterministic training outcomes	Because RL involves stochastic exploration, some runs may produce slightly different behaviors or reward trajectories
Model/Code Risks	Instability from suboptimal reward shaping or action penalties	Weak or misaligned reward components may cause oscillatory or inefficient behaviors during training
Operational Risks	Computational limitations	Minor slowdowns or environment incompatibilities may arise due to hardware or OS differences
Reproducibility Risks	Loss of transparency or traceability in experiments	Manual plotting or inconsistent analysis may hinder the reproducibility of results

Figure 5.3 Associated risks for the project.

All identified risks are low impact and fully manageable within the project’s current scope. Most mitigation strategies are already in place or under implementation, as stated in **Section 4** of the document. This ensures that any future refinements to the single-agent baseline remain stable, reproducible, and directly transferable to the upcoming multi-agent simulation phase.

6 References

- [1] D. Ge and H. Ji. “Efficient training in multi-agent reinforcement learning: A communication-free framework for the box-pushing problem. ”[Online]. Available: <https://arxiv.org/abs/2411.12246>.
- [2] H. Farivarnejad and S. Berman, “Multirobot control strategies for collective transport,” *Annual Review of Control Robotics and Autonomous Systems*, vol. 5, no. 1, pp. 205–219, Nov. 11, 2021. DOI: [10.1146/annurev-control-042920-095844](https://doi.org/10.1146/annurev-control-042920-095844). [Online]. Available: <https://doi.org/10.1146/annurev-control-042920-095844>.
- [3] K. Kondo, C. T. Tewari, A. Tagliabue, J. Tordesillas, P. C. Lusk, and J. P. How, “Primer: Perception-aware robust learning-based multiagent trajectory planner,” *arXiv (Cornell University)*, Jun. 14, 2024. DOI: [10.13140/rg.2.2.14435.57124](https://arxiv.org/abs/2406.10060). [Online]. Available: <https://arxiv.org/abs/2406.10060>.
- [4] K. Javed and R. S. Sutton. “The big world hypothesis and its ramifications for artificial intelligence. ”[Online]. Available: <https://openreview.net/forum?id=Sv7DazuCn8>.
- [5] E. Elelimy, D. Szepesvari, M. White, and M. Bowling. “Rethinking the foundations for continual reinforcement learning. ”[Online]. Available: <https://arxiv.org/abs/2504.08161>.
- [6] C. Pan et al. “A survey of continual reinforcement learning. ”[Online]. Available: <https://arxiv.org/abs/2506.21872>.
- [7] S. E. Li, *Reinforcement learning for sequential decision and optimal control*. Springer Nature Link, Jan. 1, 2023. DOI: [10.1007/978-981-19-7784-8](https://doi.org/10.1007/978-981-19-7784-8). [Online]. Available: <https://doi.org/10.1007/978-981-19-7784-8>.
- [8] R. S. Sutton and A. G. Barto, *Reinforcement learning, An Introduction*. MIT Press, Jan. 1, 1998.
- [9] S. Zhao, *Mathematical foundations of reinforcement learning*. Tsinghua University Press, Jan. 1, 2025. DOI: [10.1007/978-981-97-3944-8](https://doi.org/10.1007/978-981-97-3944-8). [Online]. Available: <https://doi.org/10.1007/978-981-97-3944-8>.
- [10] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. “Proximal policy optimization algorithms. ”[Online]. Available: <https://arxiv.org/abs/1707.06347>.
- [11] “Vertex ai custom training overview. ”[Online]. Available: <https://cloud.google.com/vertex-ai/docs/training/overview>.